

synoptic



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

End Semester Examination

Nov. 2017

Max. Marks: 100

Class: T.E.I.T

Course Code: ITC 502

Name of the Course: Operating System

Duration: 180 Minutes

Semester: V

Branch: IT

Instructions:

- (1) All Questions are Compulsory
- (2) Draw neat diagrams
- (3) Assume suitable data if necessary

Question No.		Max. Marks	CO
Q 1 (a)	Modern computers consist of processors, memories, timers, disks, mice, network interfaces, printers, and a wide variety of other devices. In the alternative view, the job of the operating system is to provide for an orderly and controlled allocation of the processors, memories, and input/output devices among the various programs competing for them. When a computer (or network) has multiple users, the need for managing and protecting the memory, input/output devices, and other resources is even greater, since the users might otherwise interface with one another. In addition, users often need to share not only hardware, but information (files, databases, etc.) as well. In short, this view of the operating system holds that its primary task is to keep track of which programs are using which resources, to grant resource requests, to account for usage, and to mediate conflicting requests from different programs and users.		
Q 1 (b)	A bootstrap is the program that initializes the operating system (OS) during startup. The term bootstrap or bootstrapping originated in the early 1950s. It referred to a bootstrap load button that was used to initiate a hardwired bootstrap program, or smaller program that executed a larger program such as the OS. The term was said to be derived from the expression "pulling yourself up by your own bootstraps," starting small and loading programs one at a time while each program is "laced" or connected to the next program to be executed in sequence.		
Q 1 (c)			



Sardar Patel Institute of Technology

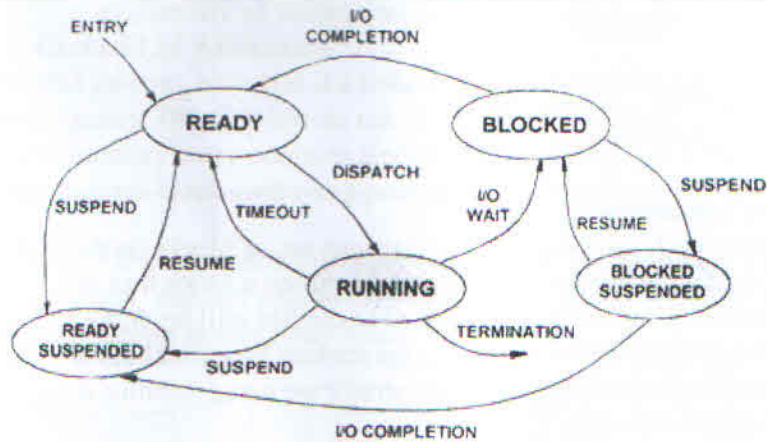
Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

	Microkernel vs Monolithic Kernel		
Q 1 (d)	<pre> int main(void) { pid_t pid = fork(); if (pid == -1) { perror("fork failed"); exit(EXIT_FAILURE); } else if (pid == 0) { printf("Hello from the child process!\n"); _exit(EXIT_SUCCESS); } else { int status; (void)waitpid(pid, &status, 0); } return EXIT_SUCCESS; } </pre>		
Q2 (a)			



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)



Q2 (b) Average Waiting Time: 6ms

OR

Schedulers - Comparison

Long Term Scheduler	Short Term Scheduler	Medium Term Scheduler
Job Scheduler	CPU scheduler	Processes swapping scheduler
Selects processes from a pool and loads them into the memory for execution	Selects those processes which are ready to execute for dispatching	Swapped out/Re-introduces the processes into memory and execution can be continued.
Controls the degree of multiprogramming	Provides lesser control over the degree of multiprogramming	Controls the degree of multiprogramming
Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between (short and long term schedulers)

Q 2 (c) Average turn Around Time for Quantum 3ms=38.2ms , 6ms= 28.4ms , 10ms= 23.4ms

As the quantum increases turnaround time decreases.



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

Q3 (a)	SSTF total seek time=262 average seek time=262/9=29.11 C-SCAN total seek time=388 average seek time=388/9=43.11																				
Q3 (b)	Contiguous Allocation In this scheme, each file occupies a contiguous set of blocks on the disk. For example, if a file requires n blocks and is given a block b as the starting location, then the blocks assigned to the file will be: $b, b+1, b+2, \dots, b+n-1$. This means that given the starting block address and the length of the file (in terms of blocks required), we can determine the blocks occupied by the file. The directory entry for a file with contiguous allocation contains - Address of starting block - Length of the allocated portion. The file 'mail' in the following figure starts from the block 19 with length = 6 blocks. Therefore, it occupies 19, 20, 21, 22, 23, 24 blocks. directory <table border="1"> <thead> <tr> <th>file</th> <th>start</th> <th>length</th> </tr> </thead> <tbody> <tr> <td>count</td> <td>0</td> <td>2</td> </tr> <tr> <td>tr</td> <td>14</td> <td>3</td> </tr> <tr> <td>mail</td> <td>19</td> <td>6</td> </tr> <tr> <td>list</td> <td>28</td> <td>4</td> </tr> <tr> <td>f</td> <td>6</td> <td>2</td> </tr> </tbody> </table> Advantages: - Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the kth block of the file which starts at block b can easily be obtained as $(b+k)$. - This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks. Disadvantages: - This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization. - Increasing file size is difficult because it depends on the	file	start	length	count	0	2	tr	14	3	mail	19	6	list	28	4	f	6	2		
file	start	length																			
count	0	2																			
tr	14	3																			
mail	19	6																			
list	28	4																			
f	6	2																			



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

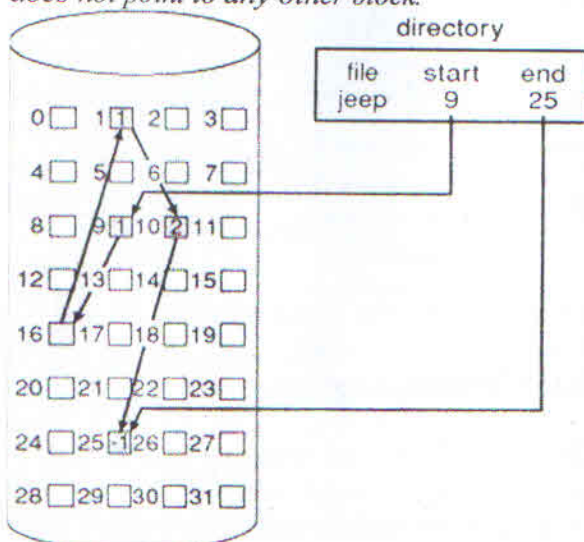
availability of contiguous memory at a particular instance.

2. Linked List Allocation

In this scheme, each file is a linked list of disk blocks which **need not be** contiguous. The disk blocks can be scattered anywhere on the disk.

The directory entry contains a pointer to the starting and the ending file block. Each block contains a pointer to the next block occupied by the file.

The file 'jeep' in following image shows how the blocks are randomly distributed. The last block (25) contains -1 indicating a null pointer and does not point to any other block.



Advantages:

- This is very flexible in terms of file size. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.
- This method does not suffer from external fragmentation. This makes it relatively better in terms of memory utilization.

Disadvantages:

- Because the file blocks are distributed randomly on the disk, a large number of seeks are needed to access every block individually. This makes linked allocation slower.
- It does not support random or direct access. We can not directly access the blocks of a file. A block k of a file can be accessed by traversing k blocks sequentially (sequential access) from the starting block of the file via block pointers.
- Pointers required in the linked allocation incur some extra overhead.

3. Indexed Allocation

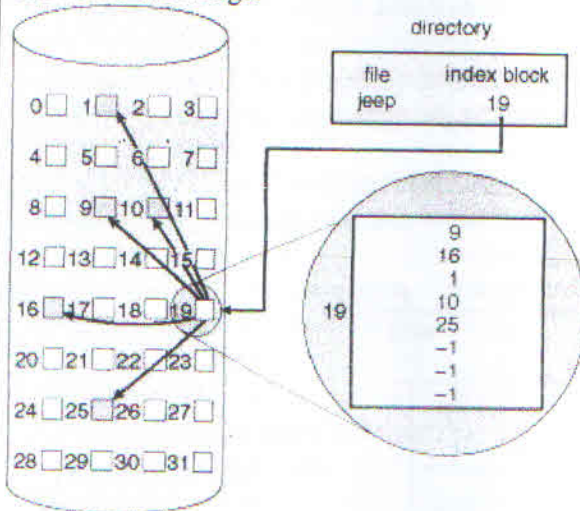
In this scheme, a special block known as the **Index block** contains the



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

pointers to all the blocks occupied by a file. Each file has its own index block. The *i*th entry in the index block contains the disk address of the *i*th file block. The directory entry contains the address of the index block as shown in the image:



Advantages:

- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.

Disadvantages:

- The pointer overhead for indexed allocation is greater than linked allocation.

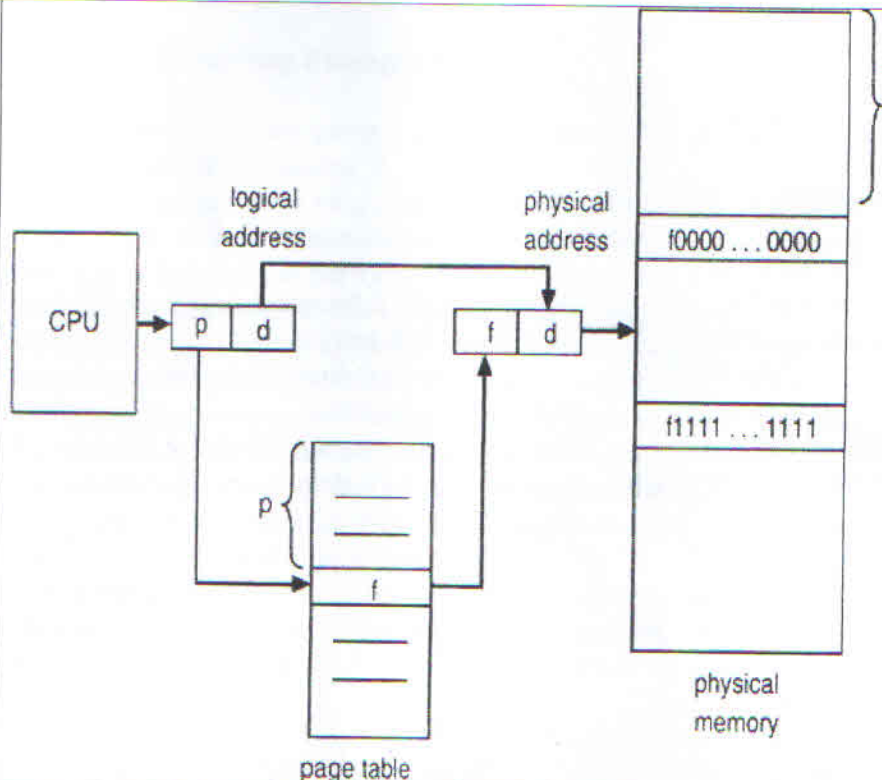
For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

Q 3 (c)



Q4 (a)

A race condition or race hazard is the behavior of an electronic, software, or other system where the output is dependent on the sequence or timing of other uncontrollable events. It becomes a bug when events do not happen in the order the programmer intended. The term originates with the idea of two signals racing each other to influence the output first.

BufferSize = N;

semaphore mutex = 1; // Controls access to critical section

semaphore empty = BufferSize; // counts number of empty buffer slots

semaphore full = 0; // counts number of full buffer slots

Producer()

{

int widget;

while (TRUE) {

make_new(widget);

down(&empty);

down(&mutex);

put_item(widget);

up(&mutex);

// loop forever

// create a new widget to put in the buffer

// decrement the empty semaphore

// enter critical section

// put widget in buffer

// leave critical section



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

```
up(&full);          // increment the full semaphore
}
}

Consumer()
{
    int widget;

    while (TRUE) {    // loop forever
        down(&full);   // decrement the full semaphore
        down(&mutex);   // enter critical section
        remove_item(widget); // take a widget from the buffer
        up(&mutex);     // leave critical section
        up(&empty);     // increment the empty semaphore
        consume_item(widget); // consume the item
    }
}
```

OR

Solution to the Critical Section Problem must meet three conditions...

1. **mutual exclusion (necessary):** if process is executing in its critical section, no other process is executing in its critical section
2. **progress (necessary):** if no process is executing in its critical section and there exists some processes that wish to enter their critical sections, then only those processes that are not executing in their remainder section can participate in the decision of which will enter its critical section next, and this decision cannot be postponed indefinitely
 - o if no process is in critical section, can decide quickly who enters
 - o only one process can enter the critical section so in practice, others are put on the queue
3. **bounded waiting (sufficient):** there must exist a bound on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - o The wait is the time from when a process makes a request to enter its critical section until that request is granted
 - o in practice, once a process enters its critical section, it does not get another turn until a waiting process gets a turn (managed as a queue)



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

	Binary and Counting Semaphore A semaphore has two parts : a counter, and a list of tasks waiting to access a particular resource. A semaphore performs two operations : wait (P) [this is like acquiring a lock], and release (V) [similar to releasing a lock] - these are the only two operations that one can perform on a semaphore. In a binary semaphore, the counter logically goes between 0 and 1. You can think of it as being similar to a lock with two values : open/closed. A counting semaphore has multiple values for count. Binary semaphore cannot handle Bounded wait as its just a variable that hold binary value. Counting semaphore It can handle bounded wait as it has converted a variable into an structure with a Queue. Strcuture implementation Binary semaphore: int s; and Counting Semaphore: Struct S { int s; Queue q; } Using the counting semaphore now process once gained the CS(Critical Section) now has to wait for the other to get the CS, so not a single process starve. Each process get a chance for CS.		
Q4 (b)	<u>Deadlock can arise if <i>four conditions</i> hold simultaneously:</u> 1) <i>Mutual exclusion(necessary)</i>: only one process at a time can use a resource 2) <i>Hold and wait (necessary)</i>: holding at least one resource and is waiting to acquire additional resources held by others 3) <i>No preemption (necessary)</i>: a resource can be released only voluntarily by the process holding it. 4) <i>Circular wait (sufficient)</i>: there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that: P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by $P_2, \dots$, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0. <u>Resource-Allocation Graph</u> A set of vertices V and a set of edges E • V is partitioned into two types: ○ $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the Processes in the system ○ $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all		



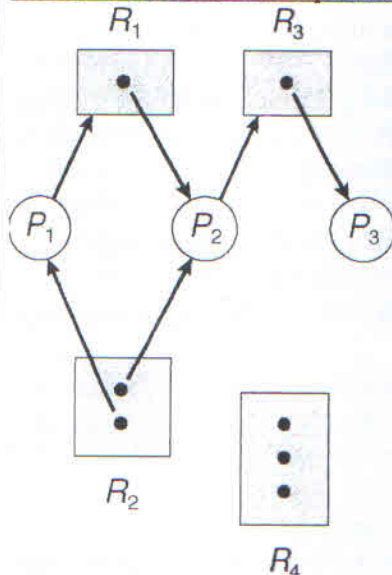
Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

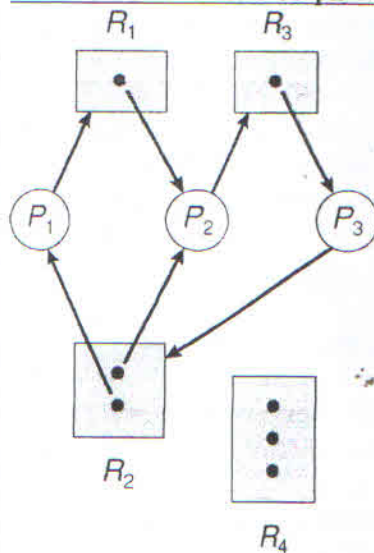
Resource types in the system

- E is partitioned into two types:
 - **request edge** – directed edge $P_i \rightarrow R_j$
 - **assignment edge** – directed edge $R_j \rightarrow P_i$

Resource Allocation Graph with *No Deadlock*



Resource Allocation Graph with *Deadlock*



Graph With A Cycle But *No Deadlock*



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

Q 4 (c)	Safe State P1-P2-P4-P0-P3		
Q5 (a)	Fixed Partitioning: Dividing the main memory into a set of non-overlapping blocks is known as fixed partition. There are two types of fixed partitioning, they are: Equal partition and Unequal Size partition. Results in internal fragmentation and can overcome by overlay technique. Dynamic Partitioning • Partitions are of variable length and number • Each process is allocated exactly as much memory as it requires • Eventually holes are formed in main memory. This is called external fragmentation • Must use compaction to shift processes so they are contiguous and all free memory is in one block Results in external fragmentation and can overcome by compaction.		
Q5 (b)	Inodes resides on the disk and the kernel reads them into an into memory which we can call as in-core inodes. Disk inodes contains the following information: - File access permissions and time (last access / modified etc.) - File ownership information. - Type of the file (regular / directory / block special / pipe) - Number of links to the file - File size and organization on disk (the file data may spread across several different and far-spaced disk location)		



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(An Autonomous Institute Affiliated to University of Mumbai)

Q 5 (c)	LRU hits: 4 miss:16 FIFO hits:5 miss:15 Frame size 4 LRU hits: 6 miss:14 FIFO hits:7 miss:13 The number of frames is equal to the size of the memory divided by the page size, increasing the page size will proportionately decrease the number of frames. Having fewer frames will tend to increase the number of page faults because of the lower freedom in replacement choice. Imagine a system with four frames with the reference history of 0, 4, 3, 1. On a page fault, LRU would victimize frame 0. With a doubling of page size the reference history becomes 0a, 1b, 1a, 0b; so the LRU victim would be 1 (corresponding to small page frames 3 and 4) when one would prefer to victimize 0a (the first half of 0).		